Since 2011

RIG
Realistic Infotech Group

Our outcomes are over 5000 trainees.

Get IT Right from RIG

Data Engineering
(Level-1)

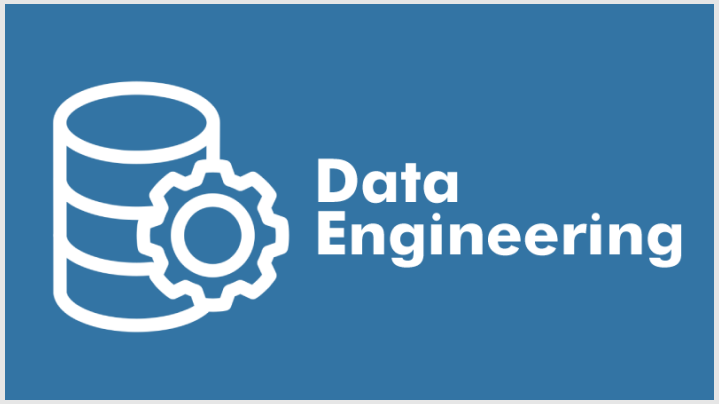
Learn, Create, and Shine

Digital Space

Level-1

- Module 1: Database Technology & Database Concepts
- Module 2: MySQL
- Module 3: Microsoft SQL
- Module 4: Oracle
- Module 5: NoSQL

Data Engineering
Module 1



Database Technology

Data Engineering
(Level-1)

Module 1: Database Technology and Database Concepts

Content

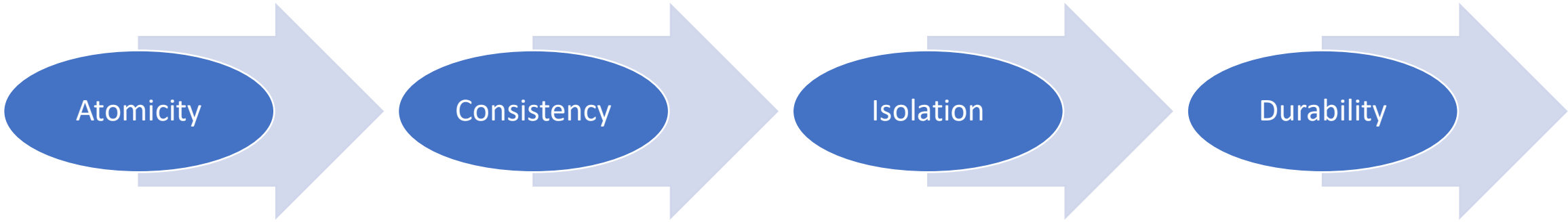
- Database Technology and Concepts
  - Types of Databases
  - Database Designs
  - Database Components
- **Relational Model**
- **Entity Relationship Model & Normalization**
- Structure Query Language (SQL)
- Database Testing

Learning Outcomes

- Understand the role and advantages of MySQL as a relational database management system (RDBMS).
- Perform backup and restore operations to protect data.
- Implement complex queries, including subqueries and nested queries.
- Understand the fundamental concepts of database technology, including types of databases (relational, NoSQL), data models, and the role of databases in data engineering.
- Compare and contrast different types of databases and their use cases.
- Understand the features and benefits of Microsoft SQL Server.
- Monitor and optimize database performance.
- Understand graph database concepts and data modeling.
- Configure backup and recovery strategies.
- Understand compliance requirements and regulatory standards related to data security (e.g., GDPR, HIPAA).
- Implement best practices for database security, including encryption, access control, and auditing.
- mplement replication, sharding, and other scalability features.

What are ACID Properties?

- ACID Properties are used for maintaining the integrity of database during transaction processing.
- ACID in DBMS stands for Atomicity, Consistency, Isolation, and Durability.



- A transaction is a single unit of operation.
- Can execute it entirely or do not execute it at all
- There cannot be partial execution.

- Once the transaction is executed, it should move from one consistent state to another.

- Transaction should be executed in isolation from other transactions (no Locks).
- During concurrent transaction execution, intermediate transaction results from simultaneously executed transactions should not be made available to each other.

- After successful completion of a transaction, the changes in the database should persist.
- Even in the case of system failures.

ACID Properties (Atomicity)

1. Atomicity (All or Nothing)
If one step fails → everything is rolled back
Customer places order but payment fails

```
START TRANSACTION;

-- Step 1: Create order
INSERT INTO orders (order_id, customer_id, total_amount)
VALUES ('ORD2001', 'C001', 200);

-- Step 2: Deduct stock
UPDATE products
SET stock = stock - 2
WHERE product_id = 'P101';

-- Step 3: Payment FAILS (simulate error)
-- e.g. invalid payment or system error

ROLLBACK;
```

- Order NOT saved
- Stock NOT reduced
- System remains unchanged
- အလုပ်တစ်ခုလုံး ပြီးရင်ပြီး၊ မပြီးရင် မူလကရှိနေတဲ့ဒေတာအတိုင်းပြန်ဖြစ်ရမယ်။

ACID Properties (Consistency)

2. Consistency (Valid Data Rules Maintained)
Database must move from one valid state → another valid state
Stock cannot go below zero

```
START TRANSACTION;

-- Check stock constraint
UPDATE products
SET stock = stock - 10
WHERE product_id = 'P102'
AND stock >= 10;

-- If no rows affected → constraint violated
-- Then rollback

ROLLBACK;
```

- Prevents invalid data (negative stock)
- Database stays consistent
- ဒေတာတွေအားလုံးစည်းကမ်းချက်တွေနဲ့ အမြဲကိုက်ညီနေရမယ်။

ACID Properties (Isolation)

Isolation (Transactions Do Not Interfere)
Two users ordering same product simultaneously
Two customers try to buy last product

```
-- Transaction A
START TRANSACTION;
SELECT stock FROM products WHERE product_id = 'P103' FOR
UPDATE;
UPDATE products
SET stock = stock - 1
WHERE product_id = 'P103';
-- WAIT (not committed yet)
-- Transaction B
START TRANSACTION;
SELECT stock FROM products WHERE product_id = 'P103' FOR
UPDATE;
-- This will WAIT until Transaction A finishes

-- After A
COMMIT;
-- Then B continues safely.
```

- No double selling
- No dirty reads
- Data integrity preserved
- အသုံးပြုသူတစ်ယောက်လုပ်နေတာကို နောက်တစ်ယောက်က နှောင့်ယှက်လို့မရဘူး။ တနည်းအားဖြင့် Record တူနေရင် တချိန်တည်းအသုံးပြုနိုင်ပါသည်။

ACID Properties (Durability)

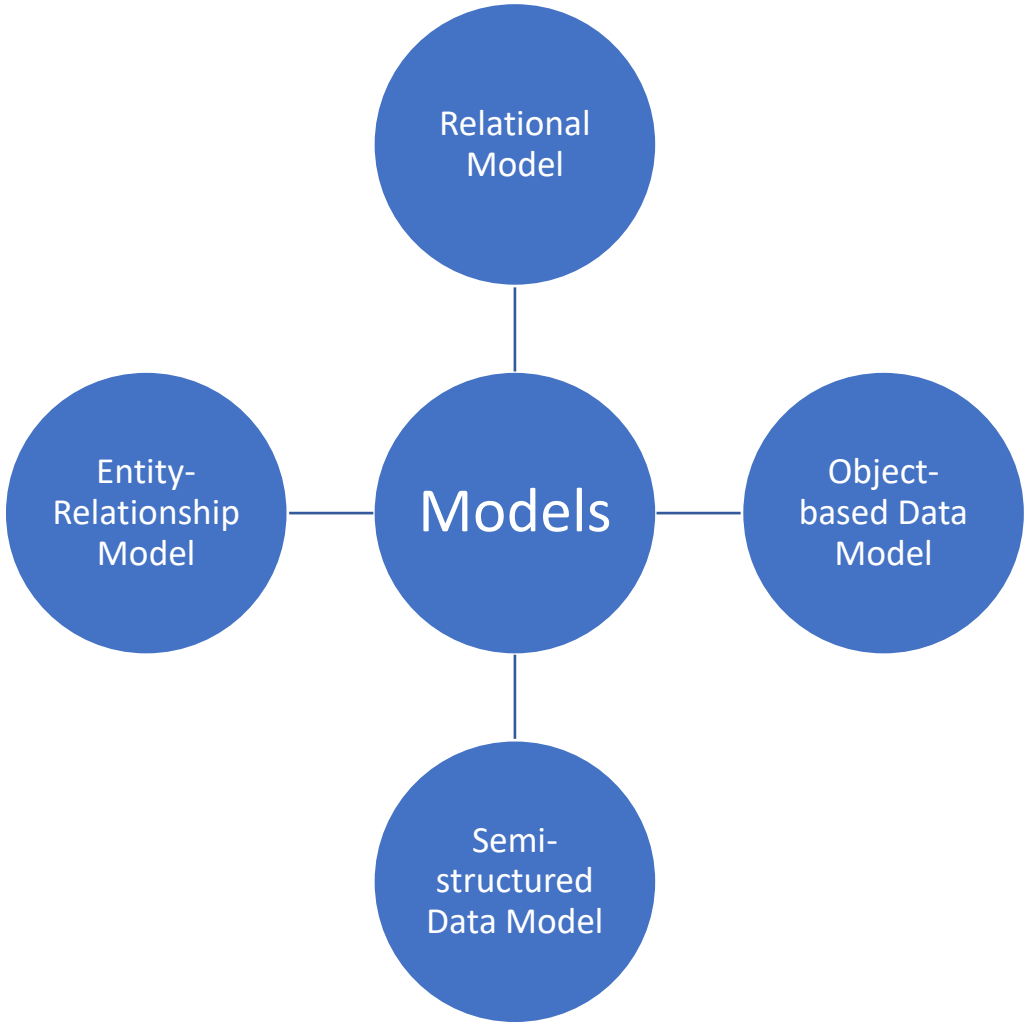
4. Durability (Permanent Storage)
Once committed → data survives crash
Order confirmed

```
START TRANSACTION;

INSERT INTO orders (order_id, customer_id, total_amount)
VALUES ('ORD3001', 'C002', 500);

COMMIT;

-- After commit:
-- Data written to disk (logs + storage)
-- Even if server crashes → order still exists
```

- Customer order is safe and permanent
- အတည်ပြုပြီးရင် တစ်ခါတည်း သိမ်းပါမယ်။သိမ်းပြီးရင် ဘယ်တော့မှ ဒေတာပျောက်မသွားရဘူး။

Data Models and Database Schema

Data Models (Conceptual/ Logical)



- After designing the conceptual model of the Database using ER diagram, we need to convert the conceptual model into a relational model.
- Relational Model can be implemented using any RDBMS language like Oracle SQL, MySQL, etc.

Relational Model

- Relational Model (RM) represents the database as a collection of relations.
- A relation is a table of values.
- Every row in the table represents a collection of related data values.
- These rows in the table denote a real-world entity or relationship.
- In the relational model, data are stored as tables.
- The physical storage of the data is independent of the way the data are logically organized.

Entity Relationship Model

- The Entity-Relationship Model is a high-level conceptual data model used to describe the structure of a database by representing:
  - Entities (real-world objects or concepts)
  - Attributes (properties of entities)
  - Relationships (how entities are connected to each other)
- Types of Normal Forms
- Normalization

- To design databases clearly
- To avoid data redundancy
- To define relationships before implementation
- To communicate database structure easily

Semi-Structure Data Model

- A Semi-Structured Data Model is a type of data model where data does not follow a fixed, rigid schema, but still contains tags, keys, or markers that help organize and interpret the data.

- No fixed table structure (schema is flexible)
- Data can vary from record to record
- Uses tags or key-value pairs
- Easy to modify and extend
- Common in modern applications (web, APIs)

JSON

```
{
  "name": "John",
  "age": 25,
  "skills": ["Python", "SQL"]
}
```

XML

```
<student>
<name>John</name>
<age>25</age>
</student>
```

Object Based Data Model

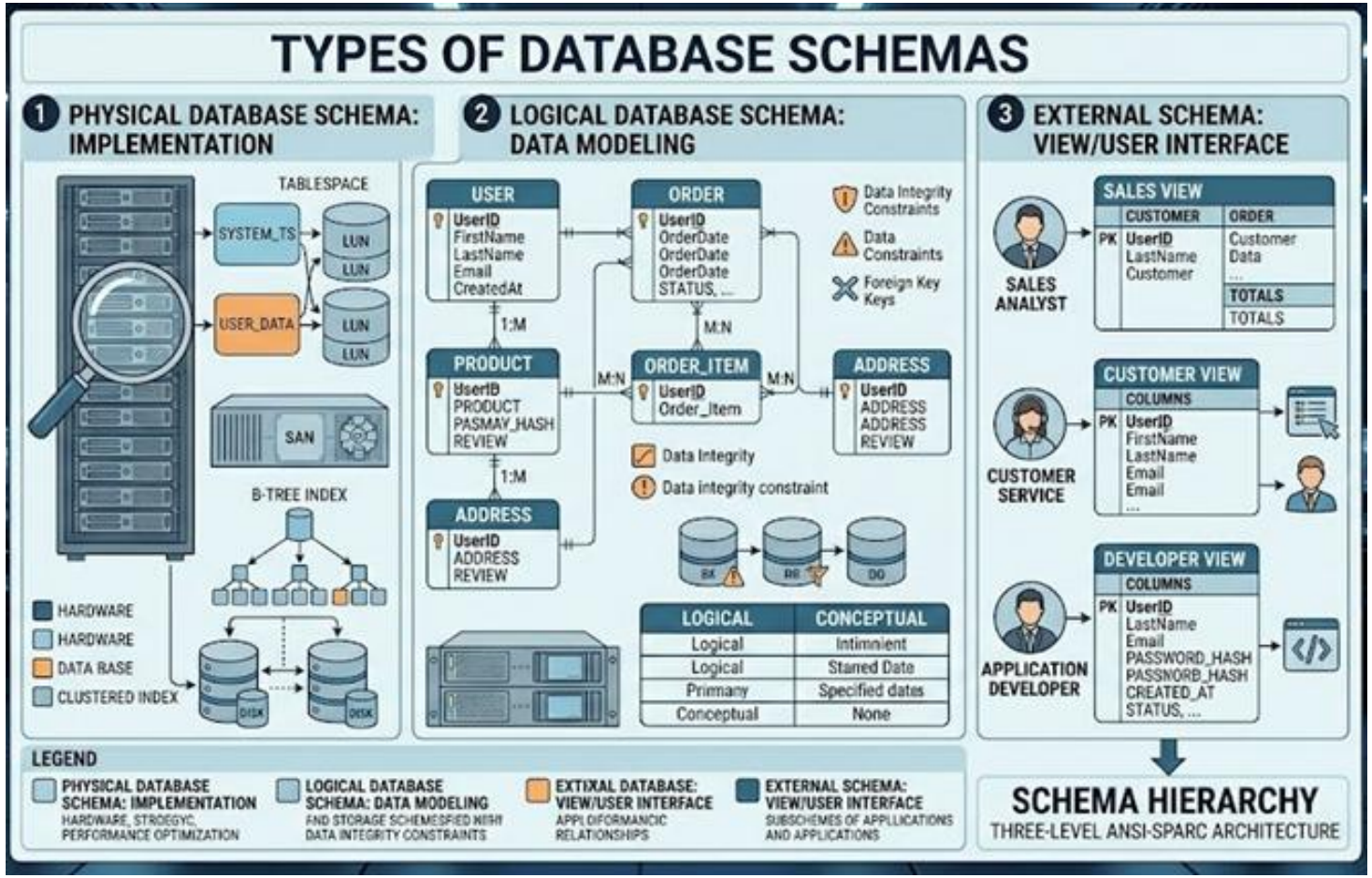
- An Object-Based Data Model is a type of data model that represents data as objects, similar to how data is handled in object-oriented programming (OOP).
- Each object contains both:
  - Data (attributes)
  - Behavior (methods/functions)

- Based on object-oriented concepts
- Supports encapsulation (data + methods together)
- Supports inheritance (reuse of objects)
- Supports abstraction
- More flexible than traditional relational models

Database Schema (Physical/Implementation)

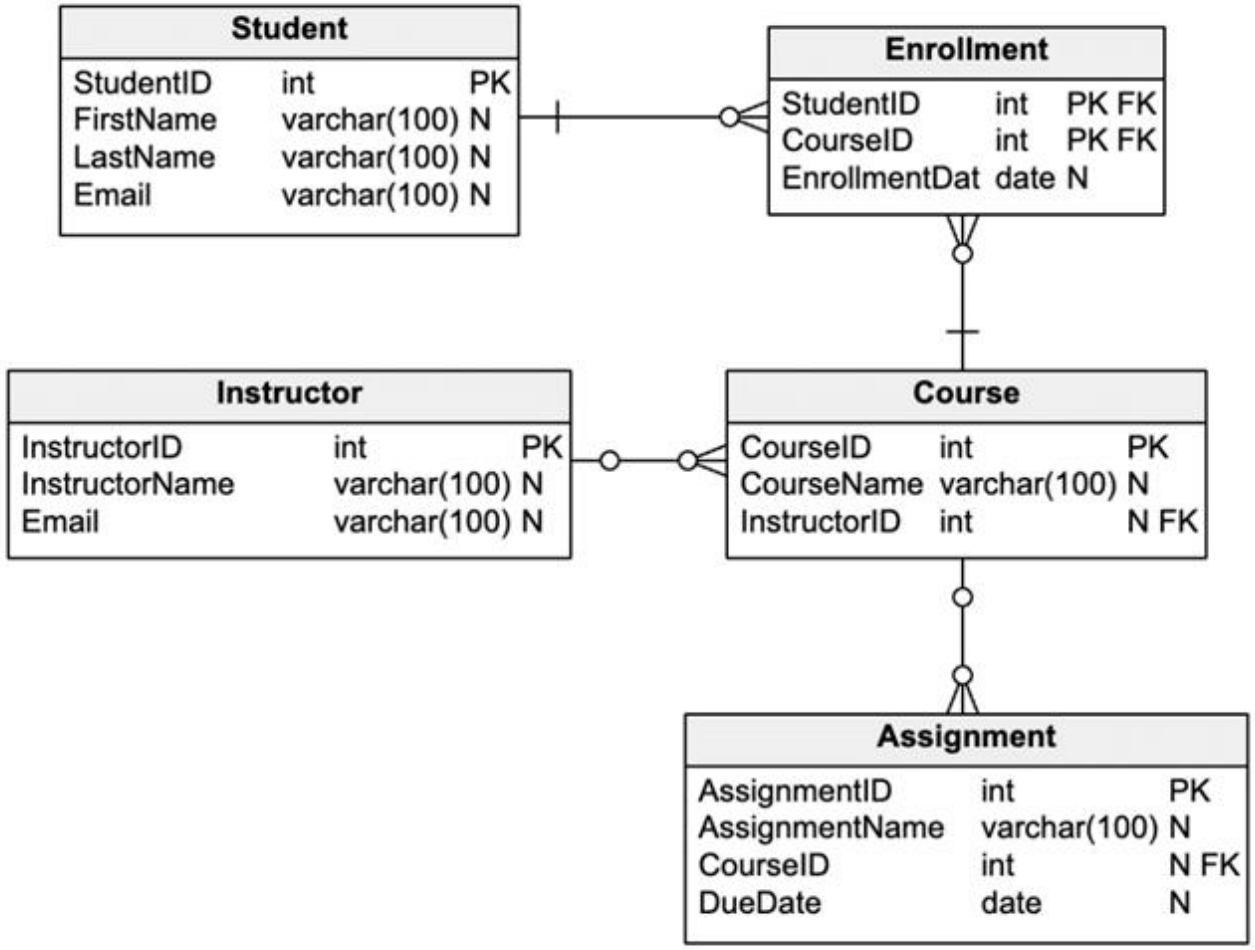
- A database schema is a logical representation of data that shows how the data in a database should be stored logically.
- It shows how the data is organized and the relationship between the tables.
- Database schema contains table, field, views and relation between different keys like primary key, foreign key.

Types of Database Schemas



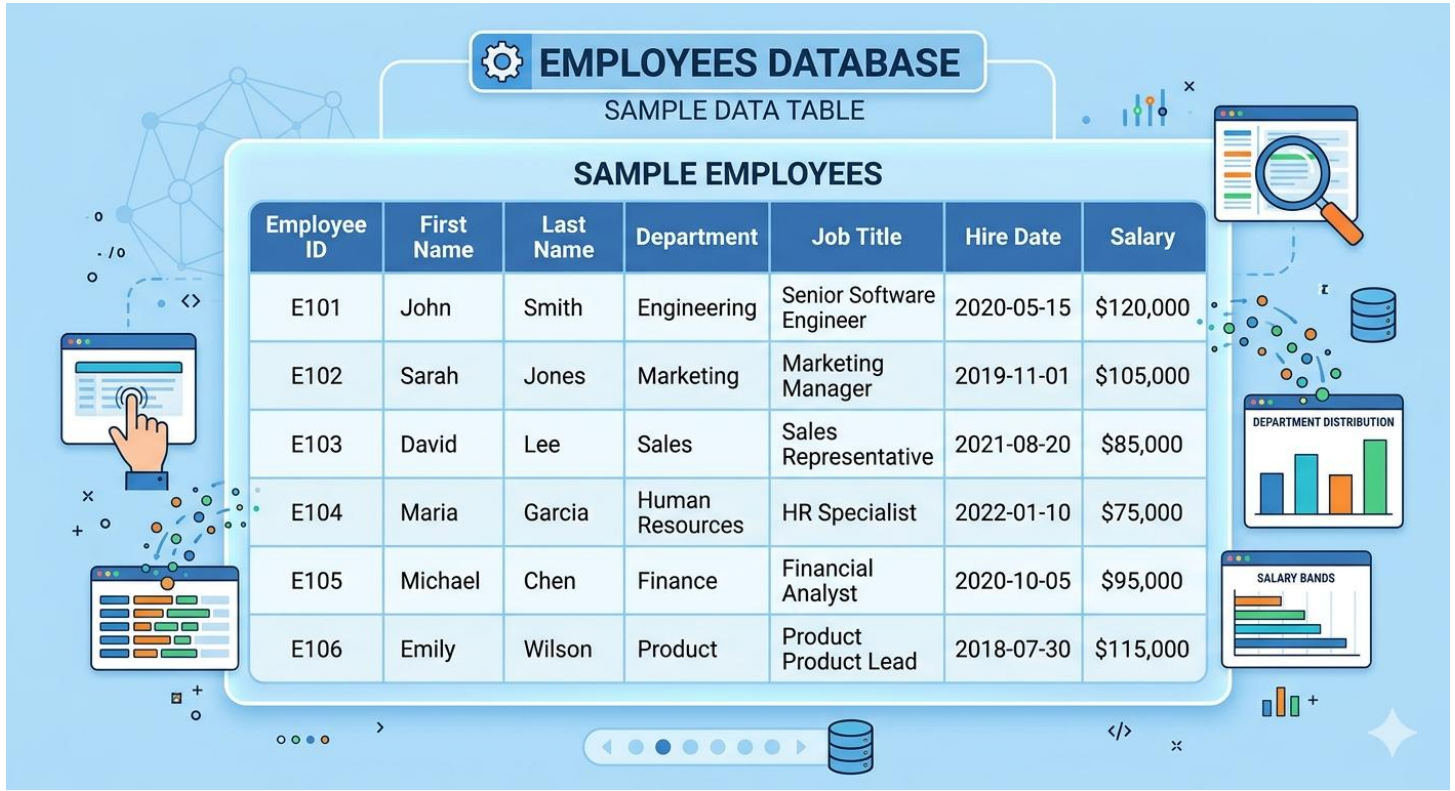
- Physical Database Schema
- Logical Database Schema
- External Schema

Schema Diagram (Student Database)



Table

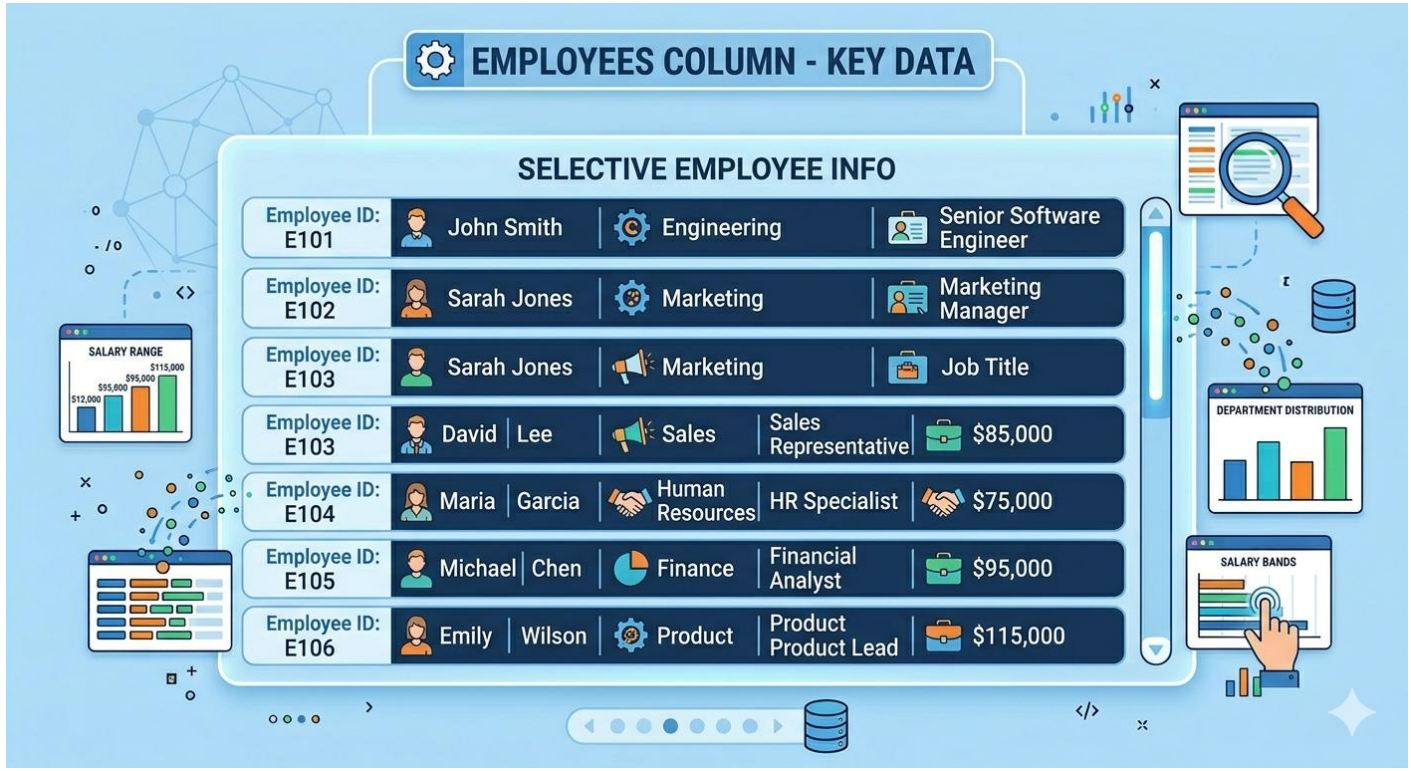
- A table is the basic structure in a database
- It organizes data into rows and columns
- Each table represents one type of data



Employee ID	First Name	Last Name	Department	Job Title	Hire Date	Salary
E101	John	Smith	Engineering	Senior Software Engineer	2020-05-15	$120,000
E102	Sarah	Jones	Marketing	Marketing Manager	2019-11-01	$105,000
E103	David	Lee	Sales	Sales Representative	2021-08-20	$85,000
E104	Maria	Garcia	Human Resources	HR Specialist	2022-01-10	$75,000
E105	Michael	Chen	Finance	Financial Analyst	2020-10-05	$95,000
E106	Emily	Wilson	Product	Product Product Lead	2018-07-30	$115,000

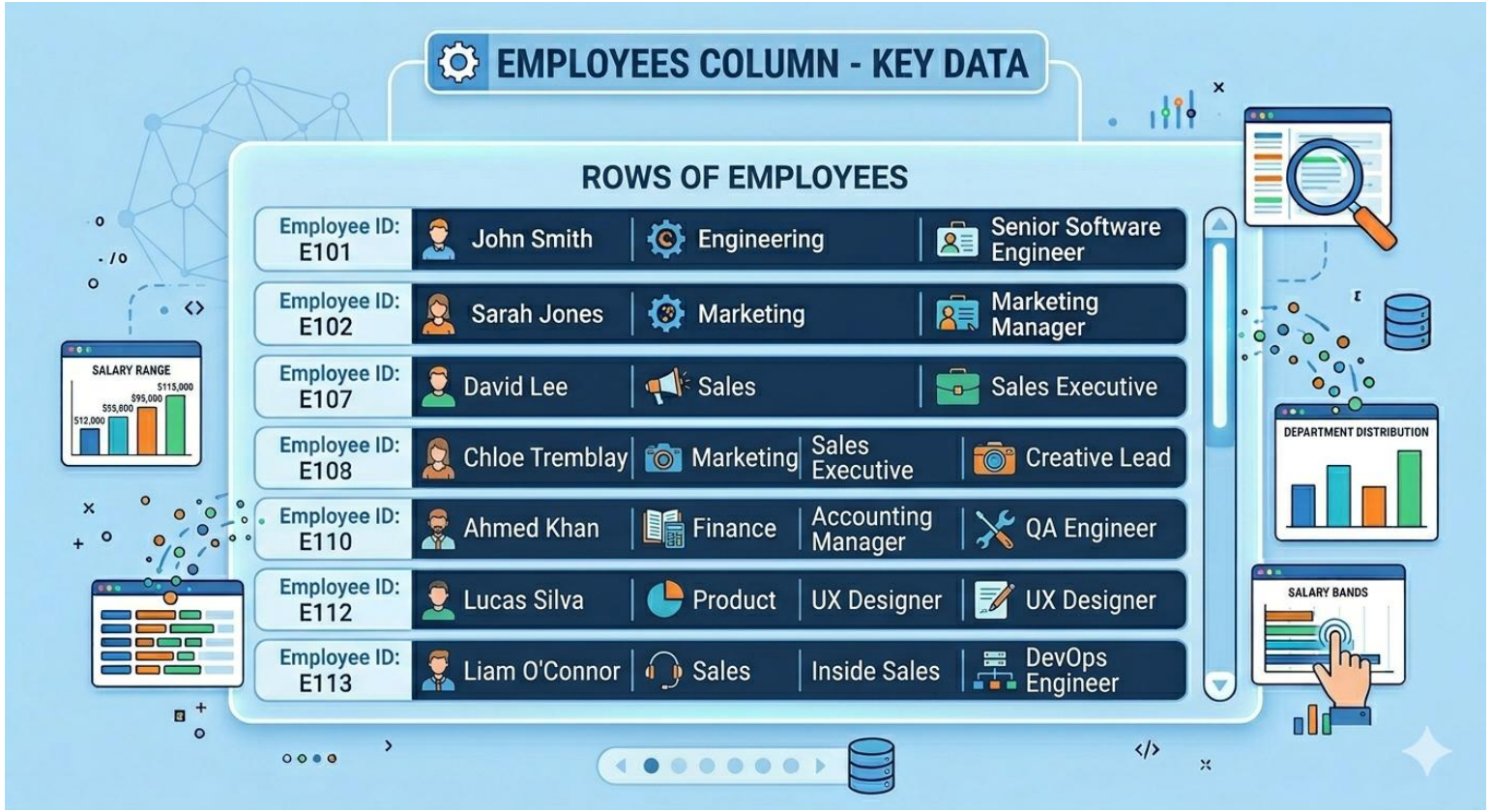
Column

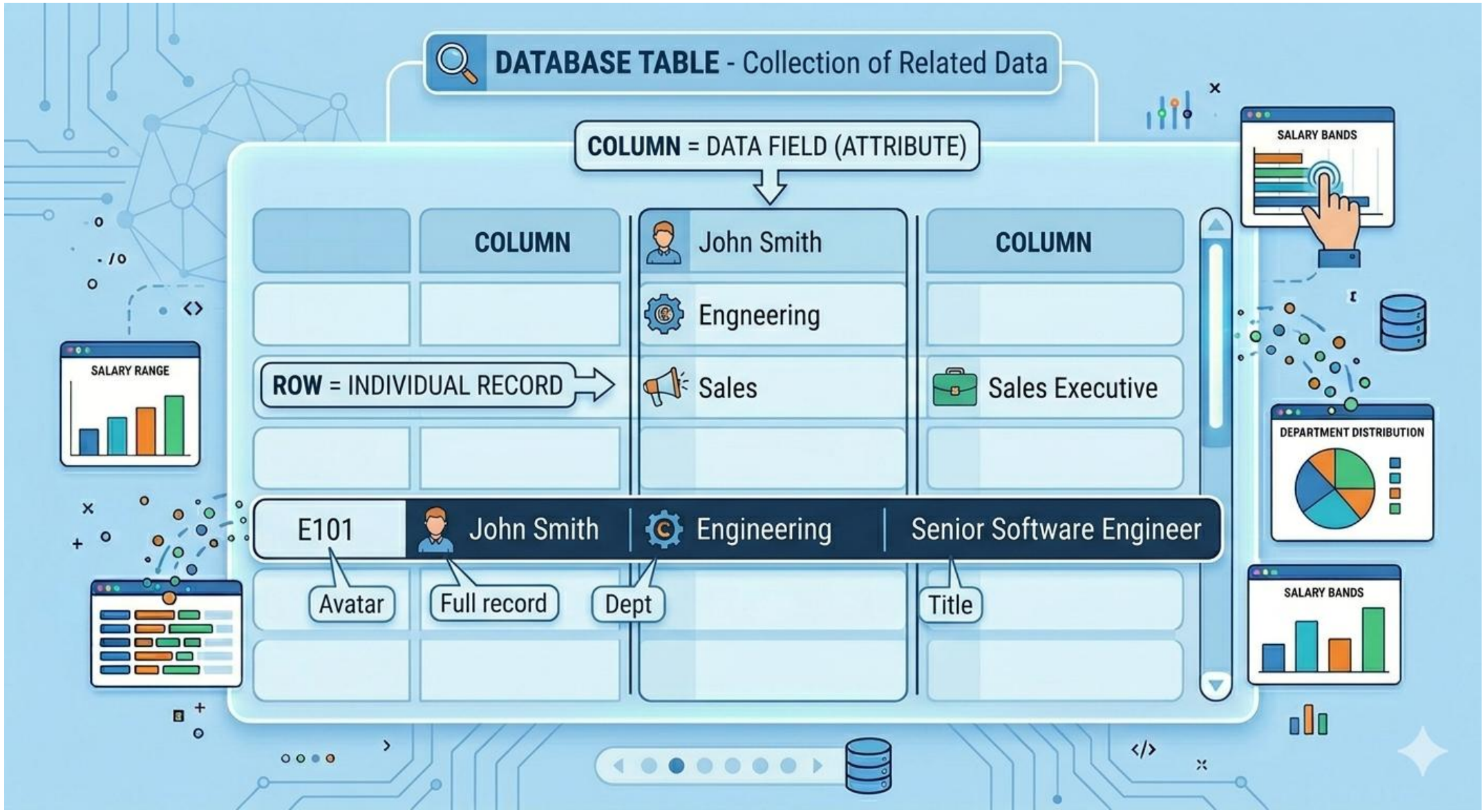
- A column represents a field (attribute)
- Defines the type of data stored



Row

- A row represents a single record
- Contains data for all columns




DATABASE TABLE - Collection of Related Data
COLUMN = DATA FIELD (ATTRIBUTE)
COLUMN
COLUMN
John Smith
Engneering
ROW = INDIVIDUAL RECORD
Sales
Sales Executive
E101
John Smith
Engineering
Senior Software Engineer
Avatar
Full record
Dept
Title
SALARY BANDS
SALARY RANGE
DEPARTMENT DISTRIBUTION
SALARY BANDS

Relational Model Concepts

Student Table

Roll_No	Name	Email	Address	Age
1	John	john@gmail.com	Yangon	18
2	Smith	smith@gmail.com	Mandalay	20
3	Amy	amy@gmail.com	Monywa	21
4	Rosy	rosy@gmail.com		19

Relational Model Concepts in DBMS

Relation Instance:

- The set of tuples of a relation at a particular instance of time is called a relation instance.
- Table 1 shows the relation instance of Student at a particular time. It can change whenever there is an insertion, deletion, or update in the database.

Roll_No
1
2
3
4

Degree:

- The number of attributes in the relation is known as the degree of the relation. The Student relation defined above has degree 5.

Cardinality:

- The number of tuples in a relation is known as cardinality. The Student relation defined above has cardinality 4.

Column:

- The column represents the set of values for a particular attribute. The column Roll_No is extracted from the relation Student.

Relational Model Concepts

NULL Values:

- The value which is not known or unavailable is called a NULL value. It is represented by blank space. e.g.; Address of Student having Roll_No 4 is NULL.

Relation Key:

- These are basically the keys that are used to identify the rows uniquely or also help in identifying tables.
- These are of the following types.

1. Candidate Key
2. Primary Key
3. Super Key
4. Foreign Key
5. Alternate Key
6. Composite Key

Candidate Key

- A Candidate Key is any column (or set of columns) that could potentially be a Primary Key because it contains unique information.
- In an Employees table, both employee_id and email_address are likely unique. Both are Candidate Keys.

Primary Keys (PK)

- A Primary Key is a unique identifier for a record in a table.
- No two rows can have the same Primary Key, and it cannot be empty (Null).

How to Use

• Auto-increment: Most developers use an integer that automatically increases by 1 for every new record.

• Immutability: Once assigned, a Primary Key should never change.

Primary Key

Properties :

- No duplicate values are allowed, i.e. Column assigned as primary key should have UNIQUE values only.
- NO NULL values are present in column with Primary key.
- Only one primary key per table exist although Primary key may have multiple columns.
- No new row can be inserted with the already existing primary key.
- Classified as: (a) Simple primary key that has a single column (b) Composite primary key has multiple columns.
- Defined in Create table / Alter table statement.
  - The primary key can be created in a table using PRIMARY KEY constraint. It can be created at two levels.
    - Column
    - Table.

Example: Student table -> Student(Roll_Number, Name, Email, Address, Age) , Roll_No is a primary key.

Foreign Keys (FK)

- A Foreign Key is a column in one table that points to the Primary Key of another table.
- This is the "bridge" that creates a relationship between two sets of data.

How to Use

• Referential Integrity: The database ensures you can't link an employee to a department_id that doesn't exist.

•Cleanup: You can set rules like "Cascade Delete," where deleting a department automatically removes or updates the records linked to it.

Composite Key

- A Composite Key is a Primary Key that consists of two or more columns combined together. This is used when a single column isn't enough to guarantee uniqueness.
- Student_id and Course_id

When to Use

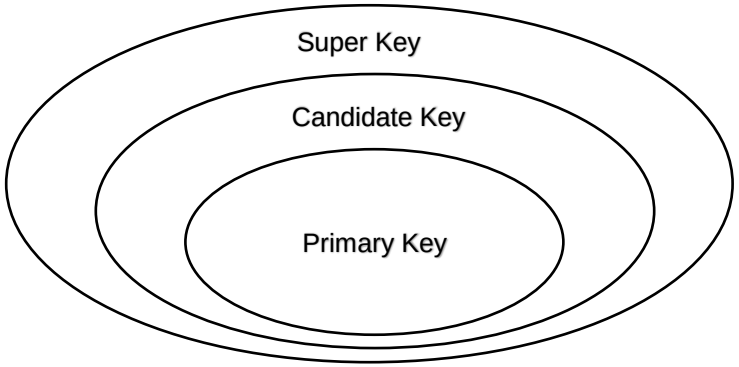
- **Junction Tables (Many-to-Many): When linking two tables together (a table that connects Students to Courses).**
- **Natural Unique Pairs: When your data has a natural relationship where two fields together create a unique record, such as Year + Quarter for financial reports.**

Super Key

The set of attributes that can uniquely identify a tuple is known as Super Key.

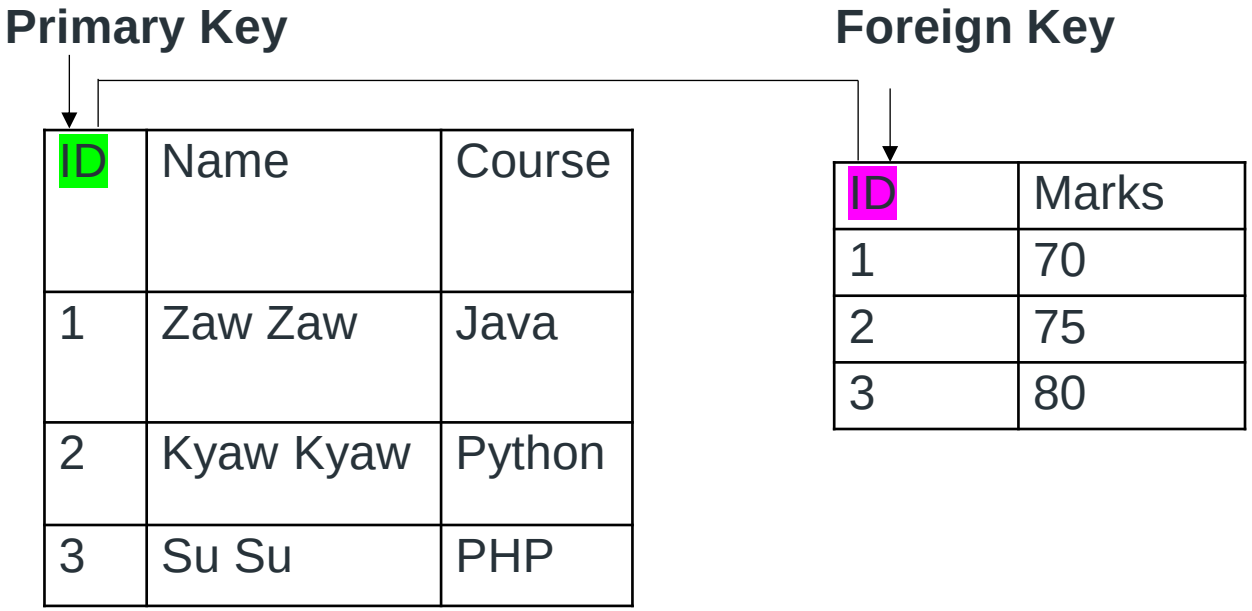
For Example, Roll_No, (Roll_No, Name), etc. A super key is a group of single or multiple keys that identifies rows in a table. It supports NULL values.

- Adding zero or more attributes to the candidate key generates the super key.
- A candidate key is a super key but vice versa is not true.
- Super Key values may also be NULL.



Relationship between Primary key, Candidate key and Super key

Example:


Primary Key
Foreign Key
ID
Name
Course
1
Zaw Zaw
Java
2
Kyaw Kyaw
Python
3
Su Su
PHP
ID
Marks
1
70
2
75
3
80
Student Table
Mark Table

Key Constraints

Every relation in the database should have at least one set of attributes that defines a tuple uniquely. Those set of attributes is called keys.

e.g: Roll_No in Student is key.

No two students can have the same roll number. So a key has two properties:

- It should be unique for all tuples.
- It can’t have NULL values.

Referential Integrity Constraints

- When one attribute of a relation can only take values from another attribute of the same relation or any other relation, it is called referential integrity.
- Suppose we have 2 relations: Student and Mark.

Student Table

ID	Name	Address	Branch_Code
1	Zaw Zaw	Pyay	CS
2	Kyaw Kyaw	Yangon	ECE
3	Su Su	Mandalay	IT
4	Phyu Phyu	Monywa	CS

Branch Table

Branch_Code	Branch_Name
CS	Computer Science
IT	Information Technology
ECE	Electronics and Communication Engineering
CV	Civil Engineering

- Branch_Code of Student can only take the values which are present in Branch_Code of Branch which is called referential integrity constraint.
- The relation which is referencing another relation is called REFERENCING RELATION (Student in this case) and the relation to which other relations refer is called REFERENCED RELATION (Branch in this case).

Entity Relationship Model

Concepts

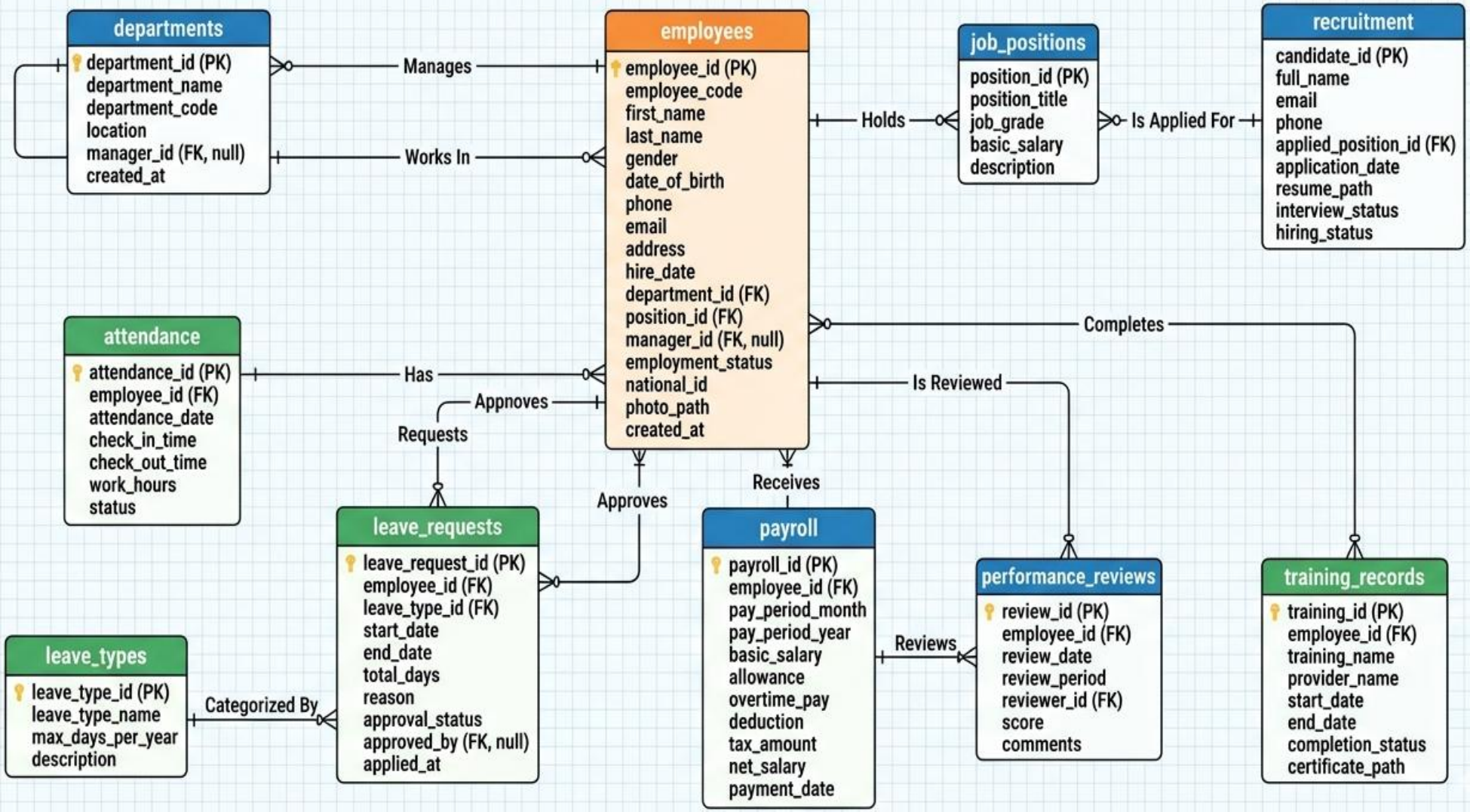
Entity Relationship Model

- Entity Relationship Model
- Entity Sets
- Types of Normal Forms
- Normalization

Entity Relationship Model

- The E-R model is very useful in mapping the meanings and interactions of real world enterprises onto a conceptual schema.
- The E-R data model employs three basic concepts: entity sets, relationship sets, and attributes.
- An E-R diagram can express the overall logical structure of a database graphically.

HR SYSTEM ER DIAGRAM



Entity Sets

- An entity is a “thing” or “object” in the real world that is distinguishable from all other objects.
- An entity set is a set of entities of the same type that share the same properties, or attributes
- Use Case: University
  - Each student in a university is an entity. An entity has a set of properties, and the values for some set of properties must uniquely identify an entity.
  - A student may have a student id property whose value uniquely identifies that student.

Normalization

- Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization rules divides larger tables into smaller tables and links them using relationships.
- The purpose of Normalization in SQL is to eliminate redundant (repetitive) data and ensure data is stored logically.

- Normalization helps to minimize data redundancy.
- Greater overall database organization
- Data consistency within the database
- Much more flexible database design
- Enforces the concept of relational integrity

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.

Types of Normal Forms

- 1NF (First Normal Form)
- 2NF (Second Normal Form)
- 3NF (Third Normal Form)
- BCNF (Boyce-Codd Normal Form)
- 4NF (Fourth Normal Form)
- 5NF (Fifth Normal Form)

Normal Form	Key Rule
1NF	Atomic values
2NF	No partial dependency
3NF	No transitive dependency
BCNF	Every determinant is a super key
4NF	No multi-valued dependencies
5NF	No join dependency redundancy

1NF (First Normal Form)

- A table is in 1NF if every field contains only one value, and there are no repeating columns or lists.
- First Normal Form (1NF) is a rule in database normalization that requires:
  - Each column must contain atomic (indivisible) values
  - No repeating groups or multi-valued attributes
  - Each record (row) must be unique (identified by a primary key)

Not in 1NF
- phone_numbers contains multiple values in one column
- Not atomic → violates 1NF

student_id	name	phone_numbers
1	Aung	091234567, 098765432
2	Su	97777777

Converted to 1NF
- Each field has **only one value**
- No repeating groups
- Table satisfies **1NF**

student_id	name	phone_number
1	Aung	91234567
1	Aung	98765432
2	Su	97777777

2NF (Second Normal Form)

- A table is in 2NF if every column depends on the whole primary key, not just part of it.
- Second Normal Form (2NF) is achieved when:
  - The table is already in First Normal Form (1NF)
  - All non-key attributes are fully dependent on the entire primary key(i.e., no partial dependency)

Not in 2NF
- product_name and price depend only on product_id, not the whole key
- This is called partial dependency

Converted to 2NF
- All attributes depend on the full primary key
- No partial dependency
- Table satisfies 2NF

Order_Details Table

order_id	product_id	product_name	price	quantity
1001	P01	Laptop	1000	1
1001	P02	Mouse	20	2

Order Table

order_id
1001

Product Table

product_id	product_name	price
P01	Laptop	1000
P02	Mouse	20

Order_Product Table

order_id	product_id	quantity
1001	P01	1
1001	P02	2

3NF (Third Normal Form)

- Every non-key column must depend only on the primary key — not on another non-key column.
- A table is in Third Normal Form (3NF) if:
  - It is already in Second Normal Form (2NF)
  - There is no transitive dependency (i.e., non-key attributes do NOT depend on other non-key attributes)

Not in 3NF
- Primary Key: emp_id
- dept_name depends on dept_id (not directly on emp_id)
- This is a transitive dependency.

Employees Table

emp_id	emp_name	dept_id	dept_name
E01	Aung	D01	HR
E02	Su	D02	Finance

Converted to 3NF
- dept_name depends only on dept_id
- No transitive dependency
- Table satisfies 3NF

emp_id	emp_name	dept_id
E01	Aung	D01
E02	Su	D02

Employees Table

dept_id	dept_name
D01	HR
D02	Finance

Department Table

BCNF (Boyce-Codd Normal Form)

- Every determinant (the left side of a dependency) must be a candidate key.
- A table is in BCNF if:
  - It is already in Third Normal Form (3NF)
  - For every functional dependency (X → Y), X must be a super key

BCNF (Boyce-Codd Normal Form)

Not in BCNF

- Assumptions:
  - A course is taught by only one instructor →course → instructor
  - A student can enroll in many courses →(student_id, course) is the primary key
- Problem:
  - course → instructor
  - But course is NOT a super key

Students Table

student_id	course	instructor
S01	DB	Dr. A
S02	DB	Dr. A
S03	AI	Dr. B

Converted to BCNF

• dept_name depends only on dept_id

•No transitive dependency

•Table satisfies 3NF

Courses Table

course	instructor
DB	Dr. A
AI	Dr. B

student_id	course
S01	DB
S02	DB
S03	AI

Enrollments Table

4NF (Fourth Normal Form)

- A table should not contain two or more independent multi-valued attributes about the same key.
- A table is in Fourth Normal Form (4NF) if:
  - It is already in Boyce–Codd Normal Form (BCNF)
  - It has no multi-valued dependencies (MVDs)
- Multi-Valued Dependency (MVD)
- MVD: X →→ Y
- Means: For a single value of X, there are multiple independent values of Y

4NF (Fourth Normal Form)

Not in 4NF
- A student can have multiple skills
- A student can have multiple hobbies
- Skills and hobbies are independent
- This creates redundant combinations

Students Table

student_id	skill	hobby
S01	Python	Chess
S01	Python	Football
S01	SQL	Chess
S01	SQL	Football

Converted to 4NF
- No redundant combinations
- Each table handles one independent relationship
- Satisfies 4NF

Student_Skills Table

student_id	skill
S01	Python
S01	SQL

student_id	hobby
S01	Chess
S01	Football

Student_Hobbies Table

5NF (Fifth Normal Form)

- If a table can be split into smaller tables without losing information, and those splits are not based on keys, then it is NOT in 5NF.
- A table is in Fifth Normal Form (5NF) if:
  - It is already in Fourth Normal Form (4NF)
  - All join dependencies are implied by candidate keys(i.e., the table cannot be losslessly decomposed into smaller tables unless it is based on keys)
- Join Dependency (JD)
- A Join Dependency exists when a table can be reconstructed by joining multiple smaller tables
- 5NF ensures:
  - No redundant data due to complex many-to-many relationships
  - Decomposition is lossless and meaningful

5NF (Fifth Normal Form)

Not in 5NF

- Assumptions:
  - A supplier can supply multiple parts
  - A project can use multiple parts
  - Supplier–Part, Supplier–Project, and Part–Project are independent relationships
- Problem:
  - Redundant combinations exist
  - Data can be derived from smaller relationships → violates 5NF

Converted to 5NF

- No redundancy
- Original table can be reconstructed using joins
- Satisfies 5NF

Supplier_Part_Project Table

supplier	part	project
S1	P1	J1
S1	P1	J2
S1	P2	J1

Supplier_Part Table

supplier	part
S1	P1
S1	P2

Supplier_Project Table

supplier	project
S1	J1
S1	J2

Part_Project Table

part	project
P1	J1
P1	J2
P2	J1

Realistic Infotech Group
IT Training & Services
No.79/A, First Floor
Corner of Insein Road and
Damaryon Street
Quarter (9), Hlaing Township
Near Thukha Bus Station
09256675642, 09953933826
http://www.rig-info.com